

Università degli Studi di Salerno
Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata

DIEM Chatbot: Progettazione e Sviluppo di un Sistema RAG Agentico per il Supporto Informativo Universitario

Relazione Tecnica — Corso di Natural Language Processing

Autori:	Alessio Bosco	a.bosco22@studenti.unisa.it – 0622702471
	Michele Apuzzo	m.apuzzo13@studenti.unisa.it – 0622702461
	Andrey Kiriyanov	a.kiriyanov@studenti.unisa.it – 0622702577
	Angelo Annunziata	a.annunziata111@studenti.unisa.it – 0622702489

Anno Accademico: 2025/2026

Indice

1	Introduzione	2
1.1	Motivazioni e Sfide	2
1.2	Contributi	2
1.3	Struttura del documento	2
2	Background e Lavori Correlati	2
3	Architettura del Sistema	3
3.1	Panoramica	3
3.2	Stack Tecnologico	3
4	Pipeline di Ingestion	4
4.1	Web Crawling Multi-Strategia	4
4.2	Parsing e Filtraggio Temporale	5
4.3	Arricchimento Contestuale: Context Header	5
4.4	Strategia di Chunking Parent-Child	6
4.5	Embedding e Indicizzazione	6
5	Core RAG Agentic	6
5.1	Evoluzione dell'Architettura: da Chain a Grafo	6
5.2	LangGraph StateGraph: Struttura del Grafo	7
5.3	Guardrail a Cascata	7
5.4	Gestione della Storia Conversazionale	8
5.5	Pipeline di Retrieval	8
5.6	Tool dell'Agente	9
5.7	Prompt Engineering	10
6	Framework di Valutazione	11
6.1	Golden Set	11
6.2	Metriche di Valutazione	11
6.3	Criteri Custom	12
6.4	Output del Runner	12
7	Validazione Intermedia e Decisioni di Stabilizzazione	12
8	Valutazione Metriche e Risultati	12
8.1	Configurazioni a Confronto	13
8.2	Risultati Aggregati	13
8.3	Analisi per Metrica	14
8.3.1	Configurazione finale (C5)	14
8.4	Affidabilità del Giudice Automatico e Re-run Correttivo	14
8.5	Riepilogo dell'Evoluzione	15
9	Discussione	15
9.1	Principali Risultati	16
9.2	Limiti del Sistema	16
9.3	Lavori Futuri	17
10	Conclusioni	17
	Riferimenti bibliografici	18

1 Introduzione

Gli atenei italiani erogano un volume crescente di informazioni attraverso portali web frammentati, spesso difficili da navigare per studenti e personale: programmi di corso, normative di ammissione, schede docente, calendario accademico e informazioni sui laboratori sono distribuite su domini separati, con strutture disomogenee e contenuti non sempre aggiornati in modo coerente. La ricerca di un'informazione specifica richiede spesso più passaggi manuali tra pagine diverse, con rischio di dati obsoleti o incompleti.

DIEMBOT nasce per rispondere a questa esigenza nel contesto specifico del Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata (DIEM) dell'Università degli Studi di Salerno. L'obiettivo è un assistente conversazionale capace di rispondere in italiano (e in inglese) a domande fattuali sul dipartimento — corsi di laurea, piani di studio, docenti, strutture, procedure amministrative — sfruttando come knowledge base i contenuti reali del sito istituzionale.

1.1 Motivazioni e Sfide

Il dominio applicativo presenta alcune sfide concrete che hanno guidato le scelte progettuali. **Eterogeneità delle fonti:** le informazioni sono distribuite su tre domini (`diem.unisa.it`, `docenti.unisa.it`, `corsi.unisa.it`) con strutture di navigazione diverse rendendo necessarie strategie di crawling differenziate. **Assenza di metadati temporali:** i CMS istituzionali non espongono segnali di data leggibili automaticamente, il che complica qualsiasi forma di recency scoring basata su documenti. **Ambiguità semantica dei chunk:** frammenti testuali come “modalità d'esame”, “ricevimento” o “obiettivi formativi” sono identici tra decine di pagine diverse: senza informazioni di contesto negli embedding, il retrieval produce falsi positivi sistematici.

A queste sfide si aggiungono requisiti di sistema tipici di un assistente universitario reale: rifiuto di domande fuori ambito, gestione di conversazioni multi-turno con risoluzione di pronomi, redazione di dati sensibili (indirizzi e-mail, numeri di carta), robustezza ad adversarial input e capacità di calcolo accademico (voti, medie TOLC).

1.2 Contributi

Il lavoro presenta i seguenti contributi originali.

1. Una pipeline di ingestion end-to-end specifica per siti universitari italiani, con crawling deterministico via sitemap, filtraggio temporale e linguistico, arricchimento contestuale via LLM (context header) e chunking Parent-Child con intestazioni semantiche incorporate in ogni chunk figlio.
2. Un'architettura RAG agentic basata su LangGraph StateGraph con otto nodi specializzati, guardrail a cascata, query rewriting automatico con iniezione dell'anno accademico e ciclo di retrieval iterativo controllato da un contatore di stato.
3. Una validazione sperimentale intermedia che ha guidato la stabilizzazione del sistema, trasformando errori osservati nelle prime run (retrieval saltato, drift multi-turno, rewrite generico, knowledge gap) in interventi mirati su grafo, prompt e valutazione.
4. Un framework di valutazione su golden set italiano con sei metriche Ragas e due criteri custom (scope-awareness e robustness).

1.3 Struttura del documento

La Sezione 2 introduce i concetti fondamentali e il lavoro correlato. La Sezione 3 descrive l'architettura generale del sistema. Le Sezioni 4 e 5 dettagliano rispettivamente la pipeline di ingestion e il core RAG agentic, inclusi tool, prompt engineering e raffinamenti del ciclo rewrite-retrieve. La Sezione 6 presenta il framework di valutazione, mentre la Sezione 7 descrive la validazione intermedia e le decisioni di stabilizzazione. La Sezione 8 analizza i risultati quantitativi sulle configurazioni C1–C5. Le Sezioni 9 e 10 discutono i limiti, i lavori futuri e le conclusioni del lavoro.

2 Background e Lavori Correlati

Il paradigma Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] supera un limite fondamentale degli LLM puri: la conoscenza cristallizzata nel training non può essere aggiornata senza un costoso re-fine-tuning. Il modello generativo riceve, oltre alla query, un contesto recuperato dinamicamente da una knowledge base esterna, riducendo le allucinazioni e permettendo aggiornamento incrementale

e citazione delle fonti. L’architettura classica combina un encoder testuale per la codifica vettoriale, un vector store per la ricerca per similarità e un modello generativo; varianti successive introducono re-ranking [Nogueira & Cho, 2020], chunking avanzato e approcci *agentic RAG* in cui il retrieval diventa una decisione dinamica del modello [Shi et al., 2023].

La granularità dei chunk è cruciale: chunk piccoli aumentano la precisione semantica ma perdono contesto, chunk grandi riducono il rumore ma penalizzano la similarity search. La strategia *Parent-Child* [LangChain, 2023] risolve il compromesso con una gerarchia a due livelli — chunk figli piccoli (≈ 400 caratteri) per il retrieval, chunk padre più ampi (≈ 2000 caratteri) restituiti al modello generativo. Il *contextual retrieval* di Anthropic [Anthropic, 2024] estende l’idea arricchendo ogni chunk con un’intestazione generata da un LLM che ne descrive il documento di origine, migliorando la disambiguazione tra frammenti semanticamente simili.

LangGraph [LangGraph, 2024] estende LangChain con pipeline definite come grafi di stato espliciti (StateGraph), superando i limiti delle LCEL chain lineari: ogni nodo opera su uno stato condiviso (TypedDict) con transizioni deterministiche o content-based, offrendo controllo fine-grained su ordine di esecuzione, gestione errori, persistenza della history via checkpointing e iniezione di nodi di sicurezza senza toccare la logica principale.

La valutazione di sistemi RAG coinvolge sia la qualità del retrieval sia quella della generazione. Ragas [Es et al., 2023] propone *Faithfulness* (affermazioni supportate dal contesto), *ResponseRelevancy* (pertinenza alla domanda) e *ContextPrecision/Recall* rispetto a una reference di ground-truth; un *LLM-as-judge* [Zheng et al., 2023] completa il quadro per criteri soggettivi come la coerenza.

I chatbot universitari hanno ricevuto crescente attenzione come strumenti di supporto agli studenti: i sistemi rule-based o a intent classification [Smutny & Schreiberova, 2020, Bansal et al., 2019] si sono rivelati insufficienti a coprire la variabilità delle domande studentesche, mentre gli LLM con RAG rendono praticabile un approccio “open-domain” ristretto al dominio. DIEMBOT si colloca in questo filone con la specificità di trattare contenuti istituzionali italiani, multi-dominio e privi di metadati temporali strutturati.

3 Architettura del Sistema

3.1 Panoramica

DIEMBOT è organizzato in tre sottosistemi principali: la **pipeline di ingestion** (raccolta, estrazione, arricchimento e indicizzazione dei contenuti), il **core RAG agentico** (grafo LangGraph per la gestione della conversazione e del retrieval) e la **UI Gradio** per l’interazione con l’utente finale. Dal punto di vista evolutivo, l’architettura è passata da una singola chain LCEL concentrata in `src/brain.py` a un grafo LangGraph modulare, con stato, nodi e orchestrazione separati nei file `src/agent/state.py`, `src/agent/nodes.py` e `src/agent/brain.py`. Questa separazione ha reso espliciti i nodi di sicurezza e ha ridotto l’accoppiamento tra guardrail, retrieval e generazione.

3.2 Stack Tecnologico

DIEMBOT è costruito su LangChain (loader, splitter, retriever, embedding), LangGraph per l’orchestrazione del grafo agentico e Chroma come vector store.

Embedding e Reranking. Due modalità via `EMBEDDING_PROVIDER`: locale (`multilingual-e5-base`) o cloud (`qwen/qwen3-embedding-8b`); `baai/bge-m3` è stato incluso tra le alternative. Il reranking è locale (`BAAI/bge-reranker-v2-m3`) o cloud (`Cohere Rerank 4 Pro`).

Modelli LLM. Due modalità via `LLM_PROVIDER`: locale via Ollama (`qwen2.5:7b` per agente e task lightweight) o cloud via OpenRouter (`gpt-oss-120b` per l’agente, `llama-3.1-8b-instruct` per guardrail/rewrite), quest’ultima la configurazione primaria. `gpt-oss-120b` è un modello open-weight a 117B parametri con architettura *Mixture-of-Experts*: attiva solo 5,1B parametri per forward pass, girando su una singola GPU H100 con quantizzazione MXFP4 nativa, e supporta profondità di reasoning interno configurabile e tool use nativo (function calling, structured output) [OpenRouter, 2026] — la combinazione di potenza di un modello di larga scala e latenza da modello molto più piccolo ha motivato la scelta come agente unico. La selezione del modello agente è stata comunque iterativa: da `qwen2.5:7b` locale, a una fase intermedia con due modelli distinti (32B routing + 9B risposta), fino al singolo

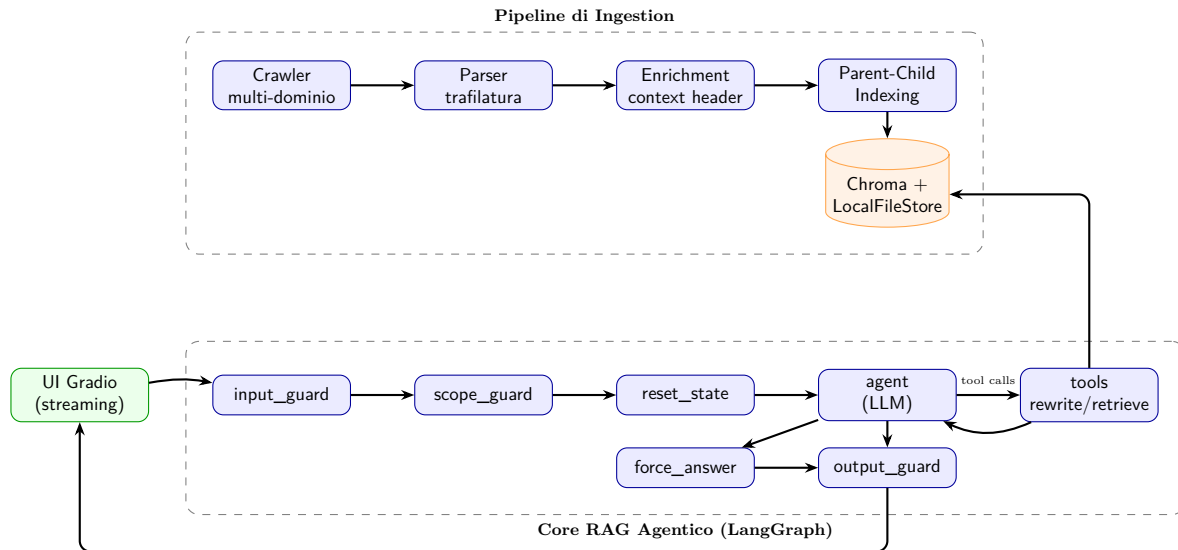


Figura 1: Architettura generale di DIEMBOT. La pipeline di ingestion (alto) popola il vector store; il core RAG agentico (basso) gestisce le interazioni conversazionali.

gpt-oss-120b finale, più stabile nel ciclo rewrite–retrieve–answer. L’enrichment (context header) usa mistralai/mistral-nemo via OpenRouter come modello primario storico, reso configurabile tramite OPENROUTER_CONTEXT_HEADER_MODEL per allineare le run finali su llama-3.1-8b-instruct.

UI. Gradio 6 (gr.ChatInterface) con streaming token-per-token; l’indice Chroma è caricato lazy all’avvio o ricostruito con il flag `-reindex`.

4 Pipeline di Ingestion

La pipeline di ingestion converte le pagine web del sito DIEM in chunk testuali arricchiti, pronti per l’indicizzazione vettoriale. È composta da quattro fasi sequenziali: crawling, parsing/filtraggio, arricchimento contestuale e indicizzazione. La traccia del progetto definiva tre domini come perimetro della knowledge base: www.diem.unisa.it (informazioni istituzionali, strutture, ricerca), docenti.unisa.it (profili, pubblicazioni, ricevimento) e corsi.unisa.it (programmi, materiale didattico). L’implementazione ha quindi dovuto gestire strutture di navigazione e CMS eterogenei con strategie di crawling differenziate.

4.1 Web Crawling Multi-Strategia

Il dominio principale diem.unisa.it viene analizzato tramite la sitemap HTML (`/?sitemap`), che individua deterministicamente i seed principali, esplorati poi con `RecursiveUrlLoader` (`max_depth=3`); questo sostituisce un crawling ricorsivo puro, non deterministico e incapace di raggiungere pagine con query string (es. dettagli laboratori). I link verso corsi.unisa.it sono estratti dall’HTML grezzo (`extract_corsi_urls`) e i PDF allegati scaricati con `load_pdfs_from_links`.

Per i docenti, il sito DIEM non linka direttamente docenti.unisa.it ma rimanda a schede rubrica.unisa.it con parametro matricola: la funzione `extract_diem_faculty_urls` risolve la matricola, verifica l’esistenza del link verso docenti.unisa.it e scarta i profili privi di pagina dedicata, evitando di includere personale di altri dipartimenti. La validazione è parallelizzata con un `ThreadPoolExecutor` (10 worker) sfruttando il rate limiter globale; ogni docente validato è poi crawlato separatamente (`max_depth=2`) confinato al proprio profilo. Analogamente, ogni corso è crawlato individualmente (`max_depth=2`), limitando `extract_corsi_urls` alla sezione `/offerta-formativa` per evitare corsi obsoleti duplicati.

In tutte le fasi, `exclude_dirs` esclude a monte risorse statiche e la funzione `filter_docs` rimuove URL con pattern indesiderati (`SKIP_DOCUMENT_SUBSTRINGS`), applica deduplica per URL canonico e hash del contenuto (≥ 350 caratteri) e scarta pagine in inglese/cinese. Il crawler usa una `RateLimitedSession` Token Bucket (5 richieste/secondo) con retry automatico (`urllib3.util.retry`); il `CrawlStateManager` mantiene un database SQLite di ETag/Last-Modified per saltare, tramite HTTP 304, il ri-download di contenuti invariati nelle re-ingestion.

La pipeline integra fonti supplementari: il modulo `easycourse.py` interroga le API di EasyCourse per calendario esami e orario lezioni dei cinque corsi DIEM, producendo un documento per coppia (CDL, insegnamento). Le risposte JSON vengono normalizzate e raggruppate per insegnamento, così un singolo retrieve restituisce appelli o lezioni rilevanti senza frammentazione. Vengono inoltre generati gli URL dei bandi DIEM (8 categorie) e scaricate le sottopagine `/focus?id=...` non raggiungibili con `max_depth=1`; una sorgente statica reinietta dati amministrativi (P.IVA, sede) rimossi dal parser come boilerplate ma richiesti da domande atomiche.

4.2 Parsing e Filtraggio Temporale

Il testo è estratto con **trafilatura** [Barbaresi, 2021] (filtro `favor_precision=True`). I PDF usano **pdfplumber**, con le pagine dello stesso documento unite prima dello split per abilitare chunk cross-page su regolamenti multi-pagina.

Trafilatura classifica erroneamente come “non-content” le pagine con struttura accordion/tabellare (pannelli corti), producendo output nulli. Per un sottoinsieme di URL critici (pagine personale, organi collegiali, commissioni, contatti corso, didattica docente) la funzione `html_extractor_for_source` attiva un'estrazione BeautifulSoup mirata via whitelist (`_is_structured_source`); per tutti gli altri URL resta trafilatura primario con fallback BeautifulSoup generico. Un parser BeautifulSoup globale è stato scartato perché la funzione di rimozione boilerplate troncava le biografie dei docenti: la whitelist mirata contiene questo rischio.

Poiché i CMS non espongono sempre metadati temporali strutturati, il filtro `filter_recent_documents` applica un cutoff minimo al 2020: se un documento contiene un anno esplicito in URL, metadati o testo analizzato, e l'anno più recente trovato è < 2020 , il documento viene scartato; in assenza di segnali temporali espliciti viene mantenuto per evitare falsi negativi su pagine istituzionali stabili. Il crawling introduce inoltre vincoli mirati a monte: gli URL dei bandi DIEM sono generati solo per anno corrente e anno precedente (nella run 2026: 2026 e 2025), mentre le pubblicazioni dei docenti sono crawlate esplicitamente per tutti gli anni dal 2020 all'anno corrente. I documenti scartati sono salvati in `dropped_temporal_filter.json` per audit.

4.3 Arricchimento Contestuale: Context Header

La pipeline di arricchimento contestuale risolve un problema strutturale del retrieval su documenti universitari: frammenti come “modalità d'esame”, “ricevimento” o “obiettivi formativi” sono semanticamente identici tra decine di pagine diverse e, senza contesto negli embedding, generano falsi positivi sistematici. La qualità dell'header è garantita da un routing ibrido *per-URL* in `enrichment.py`: per i domini con metadati titolo affidabili (`docenti.unisa.it`, `corsi.unisa.it`, EasyCourse) si usa l'euristico deterministico di `classify_context_header`; per i PDF di `corsi.unisa.it` e per le pagine di `diem.unisa.it` si invoca l'LLM (mistralai/mistral-nemo via OpenRouter, con fallback Ollama ed euristico), reso configurabile tramite `OPENROUTER_CONTEXT_HEADER_MODEL` per usare `llama-3.1-8b-instruct` nelle run finali senza modifiche al codice. La funzione `repair_context_header_semantics` corregge post-hoc classificazioni errate su URL-pattern noti; l'header è limitato a 18 parole, include un year tag [AY YYYY/YYYY] quando rilevabile e viene cachato su disco.

L'header non viene inserito nel corpo del documento ma salvato nei metadati come `context_header`: in `database.py`, dopo la generazione dei chunk padre, lo splitter `ContextHeaderTextSplitter` lo prepende a ogni chunk figlio prima dell'indicizzazione, mentre il parent resta pulito nel `LocalFileStore`. Questo corregge il limite della prima versione, dove l'header nel testo intero pre-split veniva ereditato solo dal primo chunk figlio; la validazione su seed stratificati (`simulate_enrichment.py`) ha confermato un miglioramento della qualità degli header del 93–95%. Per ridurre costo e incoerenza, un grouping per sorgente genera un solo header per URL con meno di dieci parent chunk (assegnato a tutti i chunk della pagina), mantenendo l'header per-chunk sui documenti più grandi dove sezioni diverse trattano argomenti distinti.

Esempio di context header applicato a un chunk figlio

Context header: *Corso di Ingegneria del Software – prof. Mario Rossi – orari esame a.a. 2025/2026*

Testo del chunk: *Modalità d'esame: prova scritta (60%) + orale (40%). La prova scritta si svolge in laboratorio...*

4.4 Strategia di Chunking Parent-Child

La struttura Parent-Child è implementata con i seguenti parametri (Tabella 1), scelti empiricamente per bilanciare precisione semantica dei chunk figli e ricchezza contestuale dei chunk padre. La taratura non è stata monotona: sono state provate sia configurazioni più piccole, per ridurre il rumore, sia configurazioni più grandi, per contenere schede corso e formule di regolamento in un singolo parent. La configurazione finale C5 torna a 2000/200 e 400/50 perché l'*adjacent retrieval* recupera il chunk successivo quando serve continuità, evitando di aumentare sempre la dimensione del contesto.

Tabella 1: Principali configurazioni di chunking valutate. Le dimensioni sono in caratteri.

Fase	Parent	Child	Motivazione/esito
Baseline	2000/200	400/50	Buon equilibrio iniziale, ma alcune formule PDF e schede corso restavano frammentate.
v2.1	1500/180	350/70	Chunk più densi e meno rumorosi; miglioramento su pagine list-heavy, ma perdita di contesto su regolamenti lunghi.
v3 / C4	3750/375	600/80	Copre schede corso e formule in parent più autocontenuti; aumenta però il rumore e la lunghezza del contesto.
C5 finale	2000/200	400/50	Ritorno a chunk più compatti, compensato da adjacent retrieval post-rerank.

Il `ParentDocumentRetriever` di `LangChain` viene configurato per usare il vector store `Chroma` (chunk figli) per la ricerca per similarità e il `LocalFileStore` su disco per il recupero dei chunk padre corrispondenti. Il retrieval restituisce quindi il chunk padre, più ricco di contesto, come input al modello generativo. L'indicizzazione avviene in batch di massimo 100 chunk figli per volta per gestire la memoria durante l'inserimento in `Chroma`.

4.5 Embedding e Indicizzazione

Il sistema integra la classe `OpenRouterEmbeddings` per gli embedding cloud. La versione finale usa `qwen/qwen3-embedding-8b`: il wrapper applica alle query il prefisso `Instruct: Retrieve relevant passages that answer the user's query` seguito da `Query: ...`, mentre i documenti sono indicizzati senza prefisso, seguendo il comportamento instruction-tuned del modello. In modalità locale, `E5HuggingFaceEmbeddings` mantiene la codifica simmetrica di E5 (`query:/passage:`); `e5-base` è stato usato come baseline locale perché multilingue, leggero e riproducibile senza API esterne, con embedding a 768 dimensioni gestibili in `Chroma`. L'evoluzione dell'embedding ha attraversato `multilingual-e5-small/base` (locale) e `baai/bge-m3` (cloud) prima di convergere su `Qwen3`. `BGE-M3` è stato scartato non per scarsa qualità ma perché il progetto usa `Chroma dense-only` (senza `BM25/ColBERT` che ne sfruttino la natura ibrida); `Qwen3`, pur a 4096 dimensioni, ha dato il miglior recall nelle configurazioni C3–C5.

La `similarity_score_threshold` è impostata a 0,50 con $k = 15$ candidati iniziali, poi re-scored dal reranker: rispetto a un setup locale iniziale (`e5-base`, soglia 0,70, $k = 20$), la soglia è stata abbassata perché 0,70 escludeva documenti rilevanti su query brevi, mentre k è stato ridotto a 15 dopo aver verificato che i candidati oltre quella posizione venivano quasi sempre scartati dal reranker.

5 Core RAG Agentic

5.1 Evoluzione dell'Architettura: da Chain a Grafo

L'architettura del core RAG ha attraversato quattro fasi evolutive: una *chain LCEL* a flusso fisso (`rewrite` → `retrieve` → `rerank` → `format` → LLM, nessun tool calling); un *agent + MemorySaver* con tool `retrieve/answer`, dove l'agente non chiamava sempre `retrieve` (rischio allucinazione); uno *StateGraph esplicito* con nodi separati e un `retrieve_node` obbligatorio su due modelli (32B+9B); infine l'architettura

fully agentic finale, dove il `retrieve_node` fisso è rimosso e l'agente decide autonomamente quando recuperare, con un solo modello.

La transizione chiave è l'ultima: rimuovere il nodo `retrieve_node` obbligatorio aumenta la flessibilità al costo di una LLM call aggiuntiva per turno, compensata da un hint dinamico iniettato nel system prompt quando il contatore di retrieve è zero, che forza il modello a chiamare `retrieve` come primo tool del turno senza fallback in codice.

5.2 LangGraph StateGraph: Struttura del Grafo

Il grafo è definito come un `StateGraph` `LangGraph` con stato condiviso di tipo `DiemState`:

```
class DiemState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]
    tool_call_count: int # retrieve calls only
    retrieved_context: str # last retrieved text
    last_docs: List[Document] # docs from last retrieve
```

Listing 1: Definizione di `DiemState` (`src/agent/state.py`)

I campi `tool_call_count`, `retrieved_context` e `last_docs` vengono azzerati all'inizio di ogni turno dal nodo `reset_state`, garantendo che il contatore e il contesto non si propaghino tra domande diverse.

Il grafo comprende otto nodi con le funzioni descritte nella Tabella 2.

Tabella 2: Nodi del `LangGraph StateGraph` di `DIEMBOT`.

Nodo	Funzione
<code>input_guard</code>	Controllo contenuto offensivo sull'input utente (regex + badwords).
<code>scope_guard</code>	Classificatore di ambito: DIEM/UNISA vs. fuori dominio.
<code>reset_state</code>	Azzerata contatori e contesto per il nuovo turno.
<code>agent</code>	Loop principale: invoca l'LLM con binding dei tool.
<code>tools</code>	Esegue i tool richiesti dall'agente; incrementa retrieve counter.
<code>forced_retrieve</code>	Safety net: inietta retrieve sintetico se l'agente ha generato output vuoto senza retrieve.
<code>force_answer</code>	Stub i tool calls pendenti se raggiunto <code>MAX_RETRIEVE_CALLS</code> ; forza risposta finale.
<code>output_guard</code>	Controllo offensivo + redazione PII sull'output del sistema.

Il flusso principale è:

START → `input_guard` → `scope_guard` → `reset_state`
 → `agent` ↔ `tools` → `output_guard` → END

Le deviazioni rispetto al flusso principale gestiscono i casi di rifiuto (`input_guard` o `scope_guard` iniettano un `AIMessage` e terminano), di loop forzato (`forced_retrieve` e `force_answer`) e di esaurimento del budget di retrieve (`agent` → `force_answer` → `output_guard`).

5.3 Guardrail a Cascata

Il sistema implementa tre livelli di guardrail indipendenti.

Controllo contenuto offensivo. La classe `OffensiveContentGuardrail` mantiene una lista di parole inappropriate (250+ termini, codificata in base64) e applica pattern matching regex sull'input (`input_guard`) e sull'output (`output_guard`). Il controllo è puramente regex: su un dominio testuale ristretto come quello universitario la lista è stabile e i falsi negativi trascurabili, rendendo superfluo un componente LLM dedicato.

Scope guardrail. La classe `ScopeGuardrail` implementa un classificatore gerarchico: un fast-path su circa 50 keyword del dominio DIEM/UNISA considera in-scope la domanda senza invocare l'LLM; in assenza di keyword, un modello lightweight applica un prompt binario che descrive esplicitamente le categorie da bloccare (sport, cucina, personaggi storici non accademici) e quelle da lasciar passare (persone in contesto accademico, topic ambigui). Questo disegno riduce i falsi negativi osservati su domande tipo “Chi è [nome professore]?”, inizialmente bloccate per assenza di keyword esplicite. Per i follow-up multi-turno, se esiste history precedente il nodo riscrive prima la domanda con il modello

lightweight e valuta lo scope sulla query risolta, evitando che messaggi brevi come “Davvero?” vengano bloccati solo perché privi di keyword DIEM se isolati dal contesto.

Redazione PII. La funzione `redact_pii` cerca nell’output pattern di indirizzi e-mail e numeri di carta di credito (quattro blocchi separati da spazi/trattini, per evitare falsi positivi su codici identificativi generici), sostituendo il match con un messaggio standard di privacy.

5.4 Gestione della Storia Conversazionale

La storia conversazionale è gestita da `MemorySaver` (`LangGraph`) con chiave `session_id`. Prima di passare i messaggi all’agente, il nodo `agent` applica una fase di pulizia che rimuove i `ToolMessage` dei turni precedenti, rimuove gli `AIMessage` con tool calls pendenti già risolti, e sostituisce gli `AIMessage` iniettati dai guardrail con un placeholder neutro, così che il modello veda coppie Human/AI semanticamente pulite. Questo riduce il token bloat rispetto alle versioni iniziali, dove l’accumulo di `ToolMessage` con i documenti recuperati saturava rapidamente il context window.

Gestione delle sessioni lunghe: da `summarize_history` a `sliding window`. Per le sessioni oltre i dieci turni, un primo approccio ha introdotto un tool `summarize_history` per comprimere la history in bullet list al decimo turno. I test hanno rivelato un problema di compliance: dopo la chiamata il modello generava una risposta generica senza procedere con `retrieve()`, interpretando il summary come contesto sufficiente — la sequenza multi-tool nello stesso turno risultava inaffidabile per questo modello, e il tool è stato rimosso.

La soluzione adottata è una **sliding window deterministica**: il nodo `agent` mantiene intatti gli ultimi 5 turni e rimuove `ToolMessage` e `AIMessage` con tool calls pendenti dai turni più vecchi, prima di costruire la sequenza passata al modello. Il `MemorySaver` conserva lo stato completo del grafo; la window agisce solo sulla vista ricevuta dal modello, senza round trip LLM aggiuntivo. Il sistema registra in log il numero di messaggi rimossi (`agent | sliding_window | kept_turns=5 | dropped_tool_msgs=N`).

```
human_indices = [i for i, m in enumerate(state["messages"])
                  if isinstance(m, HumanMessage)]
cutoff_idx = human_indices[-5] if len(human_indices) > 5 else 0

clean_messages = []
for i, m in enumerate(state["messages"]):
    # strip tool traffic from turns outside the 5-turn window
    if i < cutoff_idx and (
        isinstance(m, ToolMessage)
        or (isinstance(m, AIMessage) and getattr(m, "tool_calls", None))
    ):
        continue
    clean_messages.append(m)
```

Listing 2: Sliding window in `_node_agent` (`src/agent/nodes.py`): strip dei `ToolMessage` dai turni fuori finestra.

5.5 Pipeline di Retrieval

La pipeline di retrieval segue uno schema a due stadi comunemente chiamato *retrieve and rerank*.

Primo stadio — Bi-encoder. Il `ParentDocumentRetriever` `LangChain` cerca per similarità coseno nel vector store `Chroma` sui chunk figli, con $k = 15$ candidati e soglia 0,50 (`RETRIEVER_SCORE_THRESHOLD`). Il bi-encoder finale, come anticipato nella Sezione 4, è `qwen/qwen3-embedding-8b` via `OpenRouter`.

Secondo stadio — Reranker. I 15 chunk figli vengono re-scored da un secondo modello sulla coppia (query, chunk). Il reranker è dual-provider: localmente il cross-encoder `BAAI/bge-reranker-v2-m3` (singleton a livello di modulo, per eliminare un bottleneck di latenza $\approx 20s$ osservato inizialmente); in cloud `Cohere Rerank 4 Pro` (32K contesto) via `OpenRouter`, adottato per maggiore finestra di contesto e robustezza multilingue. I cinque chunk con score più alto vengono selezionati e per ciascuno viene recuperato il chunk padre corrispondente dal `LocalFileStore`.

Adjacent retrieval post-rerank. Per mitigare la frammentazione residua dei parent, ogni chunk padre riceve tre metadati (`chunk_index`, `chunk_total`, `chunk_id` hash di `source:chunk_index`); dopo il reranking, il tool `retrieve` recupera anche il chunk immediatamente successivo per ciascun parent selezionato tramite `mget`. Lo spostamento *dopo* il reranking è deliberato: il chunk adiacente è una

continuazione di un chunk già giudicato rilevante e non deve competere isolatamente nel reranker, che altrimenti potrebbe scartarlo perché privo della frase iniziale.

Recency boost. Dopo il reranking viene applicato un piccolo bonus di rilevanza basato sull'anno accademico estratto dal context header, esclusivamente sui PDF, con entità trascurabile: agisce da tiebreaker tra documenti con score quasi identico, privilegiando la versione più recente senza alterare l'ordinamento generale.

```
_YEAR_RE = re.compile(r'\[AY_\(\d{4}\)|\[(\d{4})\]\')

def _apply_recency_boost(docs):
    for doc in docs:
        year = _extract_year(doc.metadata.get("context_header", ""))
        is_pdf = doc.metadata.get("source", "").lower().endswith(".pdf")
        bonus = max(0.0, (year - 2020) * 0.001) if (year and is_pdf) else 0.0
        base = doc.metadata.get("relevance_score", 0.0)
        doc.metadata["relevance_score_boosted"] = round(base + bonus, 6)
    return sorted(docs, key=lambda d: d.metadata["relevance_score_boosted"],
                  reverse=True)
```

Listing 3: Recency boost applicato dopo il reranking (`reranker.py`).

L'anno viene estratto tramite espressione regolare dai tag `[AY 2025/2026]` o `[2021]` già presenti nel context header: nessuna chiamata aggiuntiva al modello è necessaria. Il bonus massimo è 0,005 per anno 2025, trascurabile rispetto agli score del reranker ($\in [0, 1]$).

5.6 Tool dell'Agente

L'agente dispone di due tool LangGraph, descritti nella Tabella 3. I tool `summarize` e `calculate`, presenti nelle versioni precedenti, sono stati rimossi nel corso dello sviluppo per ragioni di affidabilità e qualità descritte di seguito.

Tabella 3: Tool disponibili all'agente di DIEMBOT.

Tool	Descrizione e vincoli operativi
<code>rewrite(query)</code>	Riscrive la query in una domanda standalone risolvendo pronomi, riferimenti impliciti e query incomplete. Per contenuto accademico senza anno appende automaticamente "a.a. 2025/2026". Chiamato obbligatoriamente prima di <i>ogni</i> <code>retrieve</code> , inclusi i <code>retry</code> : al secondo tentativo produce automaticamente una query diversificata rispetto al precedente output. Non soggetto a cap.
<code>retrieve(query)</code>	Esegue il retrieval bi-encoder + reranker sulla knowledge base DIEM. Aggiorna <code>retrieved_context</code> e <code>last_docs</code> nello stato. Obbligatorio almeno una volta per turno; soggetto a cap <code>MAX_RETRIEVE_CALLS=3</code> (solo questo tool conta verso il limite).

Solo `retrieve` incrementa `tool_call_count`. Questo design deliberato permette all'agente di chiamare `rewrite` liberamente senza avvicinarsi al limite di `retrieve`, che è il collo di bottiglia computazionale.

Rimozione di `summarize` e `calculate`. Il tool `summarize(text, query)`, pensato per condensare contesti retrieved lunghi, non veniva quasi mai chiamato dal 120b, che gestiva direttamente contesti estesi. La variante `summarize_history` è stata rimossa per le ragioni discusse nella gestione della storia conversazionale; entrambe le forme di sintesi sono state sostituite da una gestione deterministica della finestra di contesto.

Il tool `calculate(ctx, op, vals)` eseguiva calcoli accademici (medie ponderate, voto di laurea) delegando al modello lightweight tramite `simpleeval`. Un benchmark su tre scenari reali (Tabella 4) ha evidenziato due problemi: sulla correttezza, `simpleeval` crashava su sintassi tuple generate dall’8b (es. `min(((PT+PC),5),5)` → “*Tuple is not available in this evaluator*”), con un secondo tentativo che produceva spesso risultati errati; sulla latenza, l’overhead del tool portava il TTFT da 15–33 s contro i 7–10 s senza tool.

```
@tool
def calculate(ctx: str, op: str, vals: dict) -> str:
    """Compute academic formulas such as weighted average or graduation grade."""
    expr = build_expression_from_context(ctx, op, vals)
    result = simple_eval(expr, names=vals)
    return json.dumps({"expression": expr, "result": result})
```

Listing 4: Schema della prima implementazione del tool `calculate`: valutazione controllata di espressioni aritmetiche generate dal modello lightweight.

Lo snippet sintetizza la versione rimossa: il modello produceva un’espressione simbolica valutata in modo controllato dal tool, isolando l’aritmetica dal modello generativo. Nella pratica, la qualità dell’espressione dipendeva ancora dal modello lightweight; quando malformata, il tool diventava un punto di fallimento invece che una garanzia.

Tabella 4: Benchmark `calculate` vs. inline per tre scenari di voto di laurea (media 27, TTFT in secondi).

Scenario	TTFT con tool	TTFT inline	Qualità
Magistrale IngInf (IE227)	15,2 s	10,3 s (−32%)	equivalente
Triennale IngInf (IE127)	33,0 s	10,2 s (−69%)	inline superiore
Magistrale IEDM (IE232)	18,5 s	7,4 s (−60%)	equivalente

Il caso triennale è stato decisivo: il tool crashava sulla sintassi tuple, il secondo tentativo produceva un valore errato con variabili nulle, mentre `gpt-oss-120b` inline restituiva la formula completa con tutti i parametri (cap, range PT, costante FC, divisore) e una tabella di scenari al variare di PT. La rimozione del tool è quindi motivata non da ragioni di semplicità, ma da affidabilità superiore e latenza ridotta.

Rewrite retry-aware. L’architettura agentic permette più retrieve nello stesso turno finché il contesto non è sufficiente, ma questa capacità risultava inutile in pratica: al retry, `rewrite` produceva una query quasi identica alla precedente, recuperando gli stessi documenti. Il tool è stato quindi esteso per rilevare il retry (controllando se esiste già un `ToolMessage` di `rewrite` nel turno corrente) e, in caso affermativo, generare una query semanticamente diversa: se la query precedente aveva aggiunto termini non richiesti (es. “DIEM”), il retry si avvicina alla formulazione originale dell’utente; altrimenti cambia livello di specificità, sinonimi o forma interrogativa. Un vincolo esplicito nel prompt (**CRITICAL: your output MUST differ from <previous_query>**) forza la diversificazione, e `rewrite()` non è mai considerato azione terminale: ogni chiamata deve essere seguita da una `retrieve`, correzione introdotta dopo aver osservato casi in cui l’agente riscriveva la domanda e poi rispondeva senza recuperare contesto.

5.7 Prompt Engineering

La progettazione dei prompt ha richiesto iterazione significativa, poiché piccole variazioni producono comportamenti qualitativamente diversi. Il sistema combina quindi più tecniche di prompting (role prompting, structured prompting via XML, few-shot e meta-prompting), ciascuna introdotta per mitigare un problema osservato empiricamente.

AGENT_SYSTEM_PROMPT. Il prompt è prevalentemente *zero-shot*: usa *role prompting*, step procedurali (*tool use*, *context validation*, *generate answer*) e tag strutturati `[KNOWLEDGE_GAP]/[FUORI_SCOPE]`, con esempi limitati a casi di ambiguità o date. Le regole operative, incluse quelle su date, ruoli accademici e calcoli multi-step, restano istruzioni nude: per il comportamento agentic procedurale è sufficiente il meta-prompting, senza esempi numerici specifici. Alcune regole derivano da errori osservati: “NEVER EMPTY” per evitare soli tag senza spiegazione, soglia knowledge-gap più morbida su contesto parziale e regola sui ruoli accademici dopo downgrade errati di docenti PO/PA a ricercatori.

REWRITE_PROMPT. Il rewrite è invece *few-shot*: regole sui casi edge (pronomi, entità da preservare, anno accademico, challenge di conferma) sono seguite da esempi input/output annotati, perché la riscrittura è un task più ambiguo e pattern-based. I casi su voto di laurea, aree di ricerca, laboratori e piano di studi sono stati aggiunti empiricamente. Tag XML-like in `AGENT_SYSTEM_PROMPT/REWRITE_PROMPT` strutturano i few-shot examples e separano dati da istruzioni.

Temperature. Il modello agente usa temperatura 0,1 per mantenere una minima flessibilità nel reasoning su task fattuali; il modello lightweight usa temperatura 0, rendendo deterministici guardrail, classificazione e riscrittura.

Gestione del voto di laurea. Il calcolo richiede parametri specifici per corso (formula FC, costanti, bonus, differenti tra i cinque corsi DIEM), gestiti con tre meccanismi: un *cancello di chiarimento* allo Step 1 (se il corso non è specificato, l'agente chiede chiarimento prima di ogni tool, per evitare un calcolo su regolamento sbagliato); un *controllo del regolamento* allo Step 2 (verifica che l'URL del documento recuperato contenga il codice del corso richiesto, altrimenti nuovo retrieve mirato). La rimozione del tool `calculate` non elimina il supporto ai calcoli accademici: il calcolo passa all'agente principale, guidato da *meta-prompting* [Suzgun & Kalai, 2024] a passaggi espliciti. Per il voto di laurea, il prompt impone di identificare prima media, FC, bonus e vincoli del regolamento, e solo dopo produrre il risultato finale.

Tag di rifiuto. I tag `[FUORI_SCOPE]` e `[KNOWLEDGE_GAP]` sono marcatori strutturati nell'output; un dizionario `_TAG_FALLBACKS` in `brain.py` mappa ciascun tag a un messaggio in italiano quando è l'unico contenuto dell'output.

6 Framework di Valutazione

6.1 Golden Set

Il dataset di valutazione (*golden set*) è composto da domande suddivise in quattro categorie progettate per coprire le modalità d'uso principali e i casi critici del sistema:

- **in_scope**: domande fattuali sul DIEM (corsi, docenti, strutture, procedure);
- **multi_turn**: sequenze di domande in cui i turni successivi contengono pronomi o riferimenti impliciti al turno precedente;
- **out_of_scope**: domande deliberatamente fuori ambito (sport, cucina, personaggi storici) per misurare la capacità di rifiuto;
- **robustness**: scenari dedicati al criterio custom descritto nella sottosezione seguente.

La valutazione finale usa il golden set italiano `golden_set_it.json`: le domande con `reference` esplicita alimentano le metriche Ragas ground-truth, le altre solo le metriche no-reference. Il dataset è stato costruito manualmente con domande realistiche sui contenuti DIEM; le categorie dedicate ai casi fuori ambito e alle challenge conversazionali sono state aggiunte in un secondo momento, dopo i primi test manuali.

6.2 Metriche di Valutazione

Il framework utilizza Ragas 0.4.3 [Es et al., 2023] con sei metriche (Tabella 5).

Tabella 5: Metriche di valutazione del framework Ragas.

Metrica	Descrizione	Reference?
ResponseRelevancy	Misura la pertinenza della risposta alla domanda.	No
Faithfulness	Proporzione di affermazioni supportate dal contesto.	No
AnswerCorrectness	Correttezza fattuale rispetto alla reference.	Sì
LLMContextPrecision	Precisione del contesto recuperato rispetto alla reference.	Sì
LLMContextRecall	Recall del contesto recuperato rispetto alla reference.	Sì
Coherence (AspectCritic)	Coerenza stilistica e logica della risposta (LLM-judge).	No

Per evitare valori NaN artificiali su righe senza reference (che Ragas tratta come errori di valutazione), il runner implementa una valutazione a due fasi: le metriche no-reference vengono calcolate sull'intera batteria di domande, mentre le metriche reference-requiring vengono calcolate esclusivamente sulle righe con campo `reference` non vuoto.

6.3 Criteri Custom

I due criteri custom sono calcolati fuori da Ragas come pass-rate sulle categorie dedicate del golden set, perché misurano comportamenti conversazionali e di sicurezza non coperti dalle metriche standard.

Scope Awareness. Per ogni domanda out-of-scope, il runner verifica se la risposta contiene un rifiuto esplicito (*strict pass*: marker [FUORI_SCOPE] o frasi standard) o un rifiuto implicito per knowledge gap (*soft pass*: il sistema dichiara di non avere informazioni senza inventare contenuto), con un LLM-judge lightweight sui casi incerti. La distinzione è metodologicamente importante: senza di essa il tasso di scope awareness risulterebbe artificialmente vicino al 100%, mascherando i casi in cui il sistema non identifica attivamente la domanda come fuori ambito.

Robustness. Per ogni scenario avversariale, il runner verifica che il sistema non cambi risposta senza motivo dopo una challenge (“Sei sicuro?”), non accetti una false premise e non esegua istruzioni di jailbreak, con override su marker strutturati quando il comportamento atteso è il rifiuto esplicito.

6.4 Output del Runner

Per ogni run, il runner produce file strutturati (`run.log`, `per_question.json`, metriche Ragas/custom e `summary.md`) e una cache `off/use/refresh` indicizzata su schema, modello, temperatura, sessione, history e domanda, evitando condivisioni tra configurazioni diverse.

7 Validazione Intermedia e Decisioni di Stabilizzazione

Le prime valutazioni hanno avuto soprattutto una funzione diagnostica: non servivano a scegliere la configurazione finale, ma a individuare i punti in cui il comportamento del sistema non era ancora stabile. Una smoke run iniziale con modello locale mostrava valori di Faithfulness nell’intervallo 0,490–0,504, incompatibili con l’obiettivo di un assistente universitario affidabile; il divario Scope Awareness strict/soft (33,3% vs 100%) indicava knowledge gap riconosciuti ma fuori ambito non marcati esplicitamente. L’analisi manuale delle risposte e dei log ha ricondotto il problema a quattro criticità principali.

Retrieve non garantito e knowledge gap. Nelle prime architetture agentiche il modello poteva rispondere senza richiamare il retrieval, oppure usare un contesto parziale come se fosse sufficiente. Per questo il grafo finale introduce un hint dinamico quando il contatore di retrieve è zero, il safety net `forced_retrieve`, tag strutturati [KNOWLEDGE_GAP] / [FUORI_SCOPE] e regole di citazione più esplicite nel prompt.

Drift nei follow-up e rewrite generico. Nei turni multi-turno, messaggi brevi come “Sei sicuro?” o domande con pronomi potevano generare un nuovo retrieve su query troppo generiche. In altri casi il modello lightweight aggiungeva meccanicamente termini istituzionali come “DIEM”, spostando il retrieval verso documenti non pertinenti. La stabilizzazione ha portato al rewrite obbligatorio prima di ogni retrieve, al retry-aware rewrite e alla risoluzione dei follow-up sulla history prima dello scope check.

Tool e alternative non adottate. Alcune soluzioni inizialmente promettenti sono state scartate perché introducevano più fragilità che benefici: i tool ausiliari di sintesi rendevano meno prevedibile il ciclo di retrieval, `calculate` aumentava la latenza e dipendeva comunque da espressioni generate dal modello lightweight, mentre `chunk` più grandi miglioravano l’autocontenimento ma aggiungevano rumore. Anche Crawl4AI è stato testato come alternativa a trafilatura: poiché le sorgenti DIEM sono esposte quasi sempre come HTML già renderizzato, il browser headless estraeva poco contenuto realmente aggiuntivo e le prestazioni end-to-end restavano comparabili, senza giustificare la maggiore complessità operativa.

Questa fase ha quindi orientato lo sviluppo verso interventi meno visibili ma più determinanti: stabilità del ciclo rewrite→retrieve, controllo della history, prompting più vincolante e valutazione più robusta. Le configurazioni C1–C5 della Sezione 8 misurano l’effetto cumulativo di queste decisioni.

8 Valutazione Metriche e Risultati

Il presente capitolo riporta i risultati della valutazione quantitativa condotta su cinque configurazioni evolutive del sistema DIEMBOT. Le prime quattro (C1–C4) introducono modifiche incremental — al modello linguistico, all’embedding, al reranker o alla strategia di chunking — per misurarne l’impatto

sulle metriche Ragas e sui criteri custom di scope awareness e robustezza. La quinta configurazione (C5) rappresenta la versione finale del sistema: stessa infrastruttura di C3 con i fix agentici successivi (rewrite retry-aware, sliding window, regulation check, academic roles, altri miglioramenti di robustezza). La valutazione usa il golden set italiano descritto nella Sezione 6: 32 domande *in_scope/multi_turn*, 8 domande *out_of_scope* e 6 scenari di robustezza. Il modello giudice è `meta-llama/llama-3.1-8b-instruct` per tutti i run.

8.1 Configurazioni a Confronto

Le cinque configurazioni rappresentano un percorso evolutivo da un sistema completamente locale (C1) a una configurazione ibrida con componenti cloud per LLM, embedding e reranking, con raffinamenti agentici incrementali (C5). La Tabella 6 ne sintetizza i parametri principali.

Tabella 6: Parametri delle cinque configurazioni valutate. *BiK*: documenti estratti dal bi-encoder al primo stadio. *CrossK*: documenti selezionati dopo il reranking. *Parent / Child*: dimensione e overlap dei chunk in caratteri.

	LLM	Embedding	Reranker	BiK / CrossK	Soglia	Parent / Child (car.)
C1	Qwen2.5-7B (locale)	e5-base 768d (locale)	cross-encoder locale	20 / 3	0.50	2000 / 200 400 / 50
C2	GPT-oss-120B (cloud)	e5-base 768d (locale)	Cohere Rerank-4-Pro	15 / 5	0.50	2000 / 200 400 / 50
C3	GPT-oss-120B (cloud)	Qwen3-Emb-8B 4096d (cloud)	Cohere Rerank-4-Pro	15 / 5	0.50	2000 / 200 400 / 50
C4	GPT-oss-120B (cloud)	Qwen3-Emb-8B 4096d (cloud)	Cohere Rerank-4-Pro	15 / 5	0.50	3750 / 375 600 / 80
C5	GPT-oss-120B (cloud)	Qwen3-Emb-8B 4096d (cloud)	Cohere Rerank-4-Pro	15 / 5	0.50	2000 / 200 400 / 50

La soglia `RETRIEVER_SCORE_THRESHOLD` è rimasta invariata a 0,50 in tutte le configurazioni.

Le transizioni sono quattro. C1→C2: LLM locale sostituito da un modello cloud di larga scala, con reranker Cohere Rerank-4-Pro e CrossK aumentato a 5. C2→C3: embedding locale a 768d sostituito da un modello cloud ad alta dimensionalità (4096d), con re-indicizzazione completa su un nuovo Chroma store. C3→C4: chunk padre +87.5% (2000→3750 caratteri) e chunk figlio +50% (400→600), per contesti più autocontenuti. C4→C5: ritorno al chunking di C3 (2000/200 | 400/50) con i raffinamenti agentici descritti nella Sezione 5 (rewrite retry-aware, sliding window, verifica del regolamento di laurea, precedenza accademica), che agiscono sul comportamento del grafo senza alterare il retrieval.

8.2 Risultati Aggregati

La Tabella 7 mostra i valori di tutte le metriche per le cinque configurazioni. I valori in **grassetto** indicano il massimo per ciascuna riga. Nessuna configurazione domina in modo assoluto: ogni transizione migliora alcune metriche a scapito di altre, evidenziando i compromessi tipici dei sistemi RAG.

Tabella 7: Valori aggregati per configurazione. *Scope Awareness* e *Robustness* sono pass rate rispettivamente su 8 e 6 scenari; le restanti metriche sono score Ragas $\in [0, 1]$. In parentesi il numero di domande con valutazione valida. † valori corretti con re-run selettivo (si veda il testo).

Metrica	C1	C2	C3	C4	C5
ResponseRelevancy	0.703 (32/32)	0.877 (32/32)	0.681 (32/32)	0.717 (32/32)	0.682 (32/32)
Faithfulness	0.764 (32/32)	0.664 (31/32)	0.696 (30/32)	0.675 (31/32)	0.752 [†] (32/32)
AnswerCorrectness	0.537 (31/32)	0.603 (30/32)	0.527 (31/32)	0.597 (30/32)	0.582 (32/32)
LLMContextPrecision	0.711 (32/32)	0.526 (32/32)	0.584 (32/32)	0.667 (31/32)	0.633 (32/32)
LLMContextRecall	0.737 (30/32)	0.797 (32/32)	0.820 (32/32)	0.799 (29/32)	0.828 (32/32)
Coherence	0.737 (19/32)	0.815 (27/32)	0.815 (27/32)	0.750 (20/32)	0.826 [†] (32/32)
Scope Awareness	0.375 (3/8)	0.500 (4/8)	0.750 (6/8)	0.500 (4/8)	0.625 (5/8)
Robustness	0.333 (2/6)	0.833 (5/6)	0.667 (4/6)	0.667 (4/6)	0.667 (4/6)

8.3 Analisi per Metrica

Sulle metriche generative, il salto da C1 a C2 (LLM cloud) porta guadagni netti su ResponseRelevancy (0.703→0.877, +24.8%) e AnswerCorrectness (0.537→0.603, +12.3%), grazie a risposte più focalizzate e accurate; il calo in C3 su entrambe (0.681 e 0.527) è attribuibile alla re-indicizzazione su un nuovo Chroma store con embedding Qwen3, che nella fase iniziale produce contesti meno centrati rispetto a un corpus ottimizzato iterativamente, mentre C4 recupera parzialmente (0.717 e 0.597) grazie a chunk più autocontenuti. Sulla Faithfulness, C1 (LLM locale, 0.764) resta la configurazione più fedele al contesto: i modelli cloud (C2–C4, 0.664–0.696) tendono a integrare conoscenza pretrainata anche in presenza di contesto esplicito.

Sulla qualità del contesto, LLMContextPrecision e LLMContextRecall mostrano traiettorie complementari: la soglia più selettiva di C1 massimizza la precisione (0.711) ma penalizza la recall (0.737), mentre l’abbassamento della soglia e l’aumento di CrossK in C2–C4, insieme all’embedding Qwen3 ad alta dimensionalità, spostano il compromesso verso la recall, con C3 al picco (0.820) e minimo di precisione in C2 (0.526). La Coherence segue un comportamento a due livelli, 0.737 con LLM locale (C1) contro 0.815 con LLM cloud (C2/C3, +10.6%), con lieve regressione in C4 (0.750) attribuibile a contesti più lunghi che introducono ridondanza nella sintesi; il campione valido per questa metrica è però ridotto (19–27 domande su 32), il che ne limita la significatività statistica.

Sui criteri custom, Scope Awareness e Robustness seguono traiettorie non monotone. Scope Awareness passa da 37.5% (C1) a un picco di 75% in C3 — l’embedding Qwen3 distingue meglio contenuto DIEM da conoscenza generica — per poi regredire a 50% in C4, dove chunk più grandi aumentano la probabilità di contenuto parzialmente fuori dominio. Robustness compie il salto più marcato dell’intera sperimentazione, da 33.3% (C1) a 83.3% (C2, +150%), per poi stabilizzarsi al 66.7% da C3 in poi: variazioni di chunking e raffinamenti agentici non influiscono più in modo significativo su questa metrica.

8.3.1 Configurazione finale (C5)

C5 condivide la stessa infrastruttura di retrieval di C3 (embedding Qwen3-Emb-8B, chunking 2000/200 | 400/50) e introduce i miglioramenti agentici descritti nel core RAG della Sezione 5. Dal punto di vista delle metriche, i risultati più rilevanti sono:

- **LLMContextRecall 0.828**: nuovo massimo assoluto (+1.0% su C3). Il principale collo di bottiglia era il nodo *rewrite*: analizzando i retrieval con `scripts/debug_retrieval.py` è emerso che le riformulazioni del modello lightweight aggiungevano meccanicamente termini generici (“DIEM”) che spostavano il risultato su documenti non pertinenti. La revisione del prompt di rewrite, il meccanismo retry-aware e la forzatura dell’ordine rewrite→retrieve hanno reso il comportamento controllato e prevedibile nella maggior parte dei casi.
- **Coherence 0.826[†]**: nuovo massimo assoluto (+1.4% su C2/C3), per due ragioni: tecnica (il re-run porta il campione valido da 27/32 a 32/32) e qualitativa (il retrieval più stabile di C5 produce contesti più pertinenti anche su domande singole).
- **Faithfulness 0.752[†]**: valore più elevato tra le configurazioni cloud (C1 locale resta prima con 0.764), grazie a regole di citazione esplicite nel prompt e alla rimozione del tool `calculate`.
- **Scope Awareness 0.625 (5/8)**: apparentemente inferiore a C3 (0.750), ma tutti e tre i fallimenti sono rifiuti corretti valutati erroneamente come scorretti dal giudice automatico.

Nota operativa sulla demo. Un vector store aggiornato all’ultima settimana di sviluppo ha mostrato un lieve calo di performance rispetto allo store consolidato (instabilità dei context header tra run, prompt/soglie ottimizzati sullo store precedente): per la demo finale si è quindi privilegiata riproducibilità usando lo store consolidato.

8.4 Affidabilità del Giudice Automatico e Re-run Correttivo

Ragas usa un LLM come giudice per *Coherence* e *Faithfulness*, con due livelli di incertezza: *strutturale* (il giudice può sbagliare la valutazione — il caso della Scope Awareness di C5, §8.3, dove tutti i fallimenti sono rifiuti corretti valutati erroneamente) e *tecnico* (su risposte lunghe il giudice può terminare anticipatamente o produrre output non parsabili, restituendo NaN e abbassando artificialmente la media). Per C5, il run iniziale ha prodotto valori depressi (Coherence 0.526, Faithfulness 0.578) per questo secondo motivo; lo script `evaluation/rerun_ragas.py` ha corretto il problema reinvoando il giudice

solo sulle coppie NaN (senza rigenerare risposte o ri-eseguire la pipeline), portando i valori a Coherence 0.826 e Faithfulness 0.752 ($\dagger$ in Tabella 7). Il re-run correttivo è stato applicato solo a queste due metriche perché lì si sono manifestati errori tecnici di parsing/terminazione; le altre metriche sono state mantenute come prodotte dal runner originale. Poiché diverse metriche Ragas dipendono comunque da valutazione automatica, i risultati vanno letti come indicatori comparativi e non come ground truth assoluta: la scelta di C5 come configurazione finale è stata quindi supportata anche da test manuali sistematici.

8.5 Riepilogo dell’Evoluzione

Le Figure 2 e 3 visualizzano l’evoluzione delle metriche Ragas e dei criteri custom sulle cinque configurazioni.

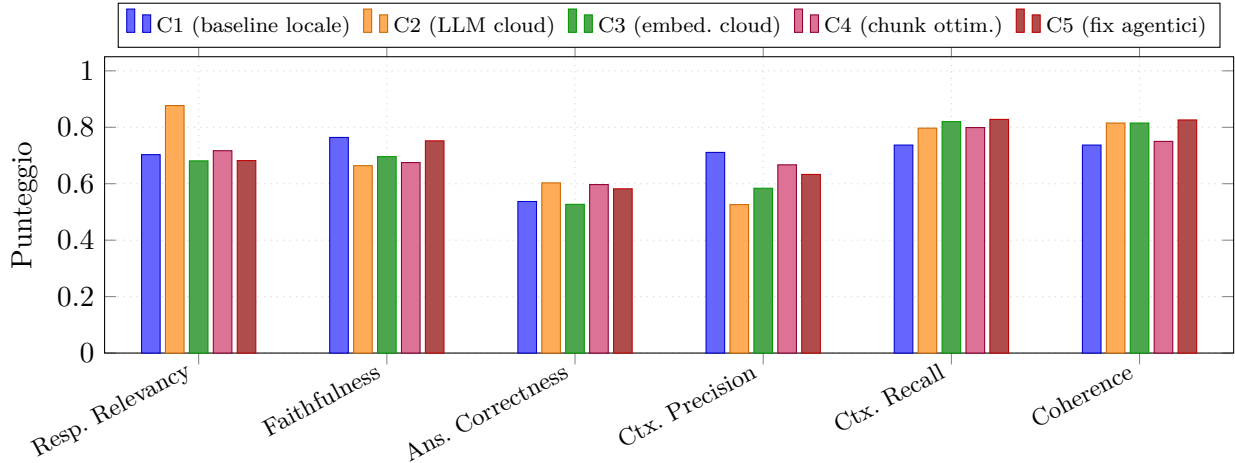


Figura 2: Metriche Ragas per le cinque configurazioni. Ogni gruppo di cinque barre corrisponde a una metrica; i colori distinguono le configurazioni. I valori di Faithfulness e Coherence per C5 sono corretti post re-run ($\dagger$).

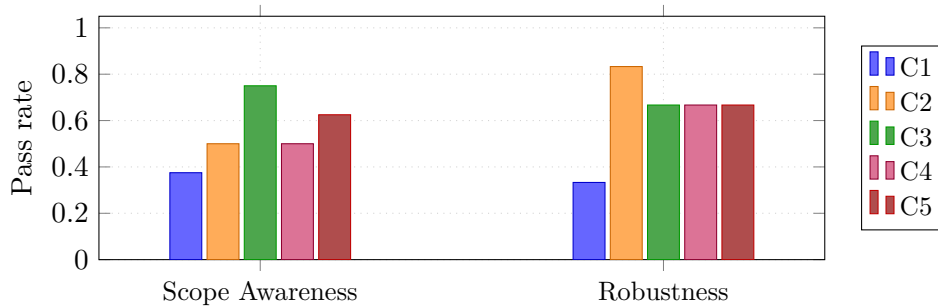


Figura 3: Pass rate per Scope Awareness (8 scenari out-of-scope) e Robustness (6 scenari avversariali) nelle cinque configurazioni.

Il passaggio dal baseline locale (C1) alla configurazione finale (C5) produce guadagni netti sostanziali su Robustness (+100%, da 33.3% a 66.7%), LLMContextRecall (+12.3%, da 0.737 a 0.828, nuovo massimo assoluto) e Coherence (+12.1%, da 0.737 a 0.826 $\dagger$). A fronte di questi guadagni si osserva una regressione su Faithfulness (−1.6%, da 0.764 a 0.752 $\dagger$) e LLMContextPrecision (−11.0%, da 0.711 a 0.633), entrambe spiegabili con l’allargamento del retrieval (soglia più bassa, CrossK maggiore) e la tendenza dei modelli di larga scala a integrare conoscenza esterna — un compromesso accettabile in un sistema orientato alla copertura e alla robustezza conversazionale. C5, come configurazione finale, raggiunge il miglior equilibrio globale tra copertura, coerenza e affidabilità agentistica.

9 Discussione

9.1 Principali Risultati

L’architettura agentic basata su LangGraph StateGraph si è dimostrata superiore alle chain lineari LCEL su più dimensioni, motivando a posteriori la scelta progettuale.

Retrieval adattivo con retry. Una chain lineare esegue il retrieve una sola volta indipendentemente dalla qualità del risultato; il grafo agentic consente invece al modello di richiamare il retrieve più volte con query progressivamente diversificate, fino a un budget controllato (`MAX_RETRIEVE_CALLS`), abilitando il meccanismo `retry-aware` descritto nella Sezione 5.

Interazione sicura a più livelli. Una chain standard non ha punti naturali per i guardrail. Il grafo esplicita otto nodi, due dedicati esclusivamente alla sicurezza (`input_guard`, `output_guard`), con `scope_guard` interposto tra input e retrieval per rifiutare domande fuori ambito con latenza minima, senza coinvolgere il modello principale.

Controllo e manutenibilità. La separazione esplicita dei nodi permette di intervenire chirurgicamente su singoli comportamenti (fix del `recursion_limit`, del nodo `forced_retrieve`, del prompt di `rewrite`, aggiunta della regola di precedenza accademica) senza destabilizzare o alterare la pipeline di retrieval sottostante.

Il sistema di context header ha prodotto un miglioramento qualitativo misurabile nella precisione del retrieval su frammenti semanticamente ambigui, applicando l’header a ogni chunk figlio senza duplicarlo nei parent restituiti al modello. La validazione intermedia ha inoltre mostrato che i guadagni più rilevanti non venivano dal cambio di estrattore, ma dalla stabilizzazione della pipeline RAG: retrieval più controllato, rewrite meno generico, history più pulita e prompt più vincolanti.

La configurazione finale C5 ha confermato che il principale collo di bottiglia del sistema, una volta consolidata l’infrastruttura di retrieval, era la stabilità del ciclo `rewrite`→`retrieve`: migliorarne la robustezza (Sezione 8.3) ha prodotto il miglior LLMContextRecall e Coherence dell’intera sperimentazione.

9.2 Limiti del Sistema

- **Assenza di metadati temporali.** I CMS istituzionali non espongono segnali di data leggibili automaticamente: il sistema non può distinguere un programma dell’a.a. 2024/2025 da uno del 2021/2022 se l’anno non è esplicito nell’URL/testo. Verificato empiricamente con `scripts/diagnose_date_tags.py` (30 pagine campionate, nessun segnale affidabile via `Last-Modified` o `<time>`). Limite strutturale del dominio, non superabile senza modifiche ai CMS.
- **Copertura della knowledge base.** Snapshot statico del sito: pagine generate solo via JavaScript senza sitemap entry e contenuti dietro login (area studenti, esse3) non sono indicizzabili.
- **Faithfulness.** Su domande senza risposta nella knowledge base, il modello tende a produrre risposte plausibili invece di `KNOWLEDGE_GAP`. Il tag e la regola di output obbligatorio riducono il fenomeno ma non lo eliminano.
- **Robustezza multi-turno.** Nelle versioni iniziali il retrieval drift al secondo turno faceva sì che una challenge (“Sei sicuro?”) producesse un re-retrieve su query diversa invece di confermare la risposta precedente. Mitigato con tre interventi: `scope_guard` riscrive i follow-up prima della classificazione, `agent` riconosce challenge brevi via pattern lessicale e forza `rewrite()`→`retrieve()`, e `REWRITE_PROMPT` trasforma la challenge nella topic query del turno precedente. Il riconoscimento resta intenzionalmente conservativo e può fallire su formulazioni lunghe o indirette.
- **Instabilità del modello di rewrite.** Il modello lightweight (8B) usato per riformulare la query è strutturalmente meno affidabile su query specializzate, polisemiche o molto brevi, con riformulazioni errate o generiche che degradano silenziosamente il retrieve. Retry e forzatura dell’output (C5) mitigano senza eliminare il fenomeno; un modello più capace per il solo nodo `rewrite` resta un’ottimizzazione aperta.
- **Auto-valutazione della pertinenza.** Lo Step 2 chiede all’agente di verificare se il contesto risponde alla domanda, ma la valutazione è eseguita dallo stesso LLM che userà il contesto: su casi *topic-adjacent* (es. “programma del corso di Ingegneria del Software” che atterra su piano di studi invece che scheda insegnamento) il modello può essere troppo ottimistico. Servirebbero segnali strutturati più forti (context header più discriminanti) o un judge di pertinenza separato.
- **Completezza su enumerazioni lunghe.** I cinque chunk padre selezionati dal reranker possono non coprire liste molto estese (es. strumentazione del Laboratorio MIVIA), rendendo impossibile una

risposta esaustiva in un turno. Mitigabile aumentando CrossK/dimensione chunk (a costo di rumore e latenza) o affinando la query verso una sottocategoria.

- **Latenza di risposta.** In uso interattivo la media è circa 30 secondi, con tempi maggiori per retrieval multiplo e ragionamento numerico (es. voto di laurea).
- **Limiti di lettura dei PDF.** Estrazione testuale, non OCR visuale: tabelle e PDF scansionati possono produrre testo disordinato o illeggibile, motivando OCR dedicato.
- **Query multi-topic.** Domande che combinano due richieste distinte (es. elenco laboratori + attrezzature di uno specifico) portano `rewrite()` a privilegiare la parte più generale, con retrieve insufficiente per la seconda sotto-domanda; mitigabile suddividendo in due turni, o con un nodo di query decomposition non implementato nella versione finale.

9.3 Lavori Futuri

- **Modelli più capaci per enrichment e rewrite.** I nodi di context header e rewrite usano un modello lightweight da 8B, fonte di instabilità su query ambigue (Sezione 9.2); un modello più capace migliorerebbe direttamente la precisione del retrieval senza alterare l'infrastruttura.
- **Fine-tuning sul dominio DIEM.** Un fine-tuning supervisionato del modello agente su conversazioni reali, domande del golden set e correzioni manuali specializzerebbe lo stile di risposta e ridurrebbe gli errori su entità specifiche (docenti, corsi, regolamenti), senza sostituire il RAG.
- **Human feedback e apprendimento continuo.** Il sistema non apprende dalle interazioni: feedback espliciti (*thumbs up/down*) e impliciti (follow-up di correzione) potrebbero alimentare un dataset di preferenze per RLHF/DPO, il riordino dei documenti recuperati o regression test mirati.
- **Hybrid search (BM25 + dense).** Una componente BM25 sparse affiancata al bi-encoder Qwen3 migliorerebbe il recall su query con termini tecnici esatti (codici corso, acronimi) non sempre catturati dalla sola similarità semantica.
- **OCR avanzato per PDF.** Modelli OCR vision-language dedicati (es. GLM-OCR) consentirebbero di indicizzare PDF scansionati o a layout complesso oggi non accessibili all'estrattore testuale.
- **Interfaccia vocale.** L'integrazione di TTS (es. ElevenLabs API) amplierebbe l'accessibilità senza modifiche al core RAG.
- **Crawling con agent browser.** Qualora venissero incluse nuove sorgenti realmente dinamiche, un agent browser potrebbe integrare il crawler HTML statico per accedere a contenuto JavaScript, accordion e sezioni lazy-loaded non indicizzabili.

10 Conclusioni

Questo lavoro ha presentato la progettazione, l'implementazione e la valutazione di DIEMBOT, un sistema RAG agentic per il supporto informativo universitario applicato al Dipartimento DIEM dell'Università di Salerno. Il percorso tecnico ha attraversato cinque configurazioni evolutive — da un baseline locale a un grafo LangGraph con otto nodi specializzati e una pipeline di retrieval cloud — motivate da esigenze concrete emerse durante i test: rifiuto affidabile delle domande fuori ambito, gestione robusta della conversazione multi-turno, stabilità del ciclo `rewrite`→`retrieve` e fedeltà delle risposte al contesto.

I contributi principali sono: (1) una pipeline di ingestion specifica per siti universitari italiani con crawling resiliente, caching incrementale, filtro temporale multi-tier, parsing strutturato mirato e context header semantici su ogni chunk figlio; (2) un'architettura agentic con guardrail a cascata, dual-provider per embedding/reranker/LLM e budget controllato di retrieval; (3) meccanismi di robustezza agentistica (`rewrite` retry-aware, sliding window, regulation check, academic roles) che stabilizzano il comportamento senza alterare l'infrastruttura di retrieval; (4) l'integrazione di dati real-time (EasyCourse) nella knowledge base; (5) una validazione intermedia che ha trasformato criticità osservate nelle prime run in decisioni architetturali mirate.

La configurazione finale (C5) raggiunge LLMContextRecall 0.828 e Coherence 0.826 — entrambi i massimi assoluti dell'intera sperimentazione — con Faithfulness 0.752 e Robustness 66.7%. I limiti residui più rilevanti sono l'instabilità strutturale del modello lightweight utilizzato nel nodo `rewrite` e l'impossibilità di garantire completezza su contenuti enumerativi che superano la finestra di retrieval. Le direzioni di sviluppo prioritarie identificate (modelli più capaci per `rewrite`, hybrid search BM25+dense, OCR avanzato) indirizzano direttamente questi limiti.

Riferimenti bibliografici

- Anthropic (2024). *Contextual Retrieval*. Technical blog post, Anthropic, September 2024.
- Barbarese, A. (2021). Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of ACL 2021*, pages 122–129.
- Bansal, H. and Khan, R. (2019). A review paper on human computer interaction. *International Journal of Advanced Research in Computer Science and Software Engineering*, 8(4):53–56.
- Es, S., James, J., Anke, L. E., and Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.
- LangChain (2023). *Parent Document Retriever*. LangChain documentation, version 0.3.
- LangGraph (2024). *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. LangChain Inc., 2024.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.
- Nogueira, R. and Cho, K. (2020). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., Zettlemoyer, L., and Yih, W. (2023). REPLUG: Retrieval-augmented language model pre-training. *arXiv preprint arXiv:2301.12652*.
- Smutny, P. and Schreiberova, P. (2020). Chatbots for learning: A review of educational chatbots for the Facebook Messenger. *Computers & Education*, 151:103862.
- OpenRouter (2026). OpenAI: gpt-oss-120b – API pricing & benchmarks. <https://openrouter.ai/openai/gpt-oss-120b>.
- Suzgun, M. and Kalai, A. T. (2024). Meta-prompting: Enhancing language models with task-agnostic scaffolding. *arXiv preprint arXiv:2401.12954*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.